

Algoritmos y Estructuras de Datos.

- Clase 1 - Parte 2 - Contenedores -

Cuerpo Docente: Dr. Diego Agustín Ambrossio.

Material adaptado de la cátedra "Introduction to Programming" de la Université du Luxembourg. Gracias a mis estimados colegas Dr. Giovanni Casini (ISTI-CNR) y Dr. Xavier Parent (TU Wien).

En la lección anterior hemos visto:

- Algunos tipos de datos basicos:
 - Enteros Integers (int)
 - Números Reales (float)
 - Valores Booleanos (bool)
 - Cademas de caracteres (strings, 'str')
- Algunas Funciones:
 - input
 - print
- Condicionales: **if, elif, else**

Hoy veremos los siguiente:

- Diferencias entre *funciones* y *métodos*
- Más de Tipos de Datos:
 - Enteros Integers (int)
 - Números Reales (float)
 - Valores Booleanos (bool)
 - Cademas de caracteres (strings, 'str')
 - Tuplas (tuple)
 - Conjuntos (set)
 - Diccionarios (dict)
 - Listas
- Definición y uso de Rangos
- Ciclos y Repeticiones **for, while**

Preliminares: Funciones vs. Métodos

En Python tenemos dos tipos diferentes formas de retornar valores o realizar acciones, en base a parámetros enviados. Podemos llamar a una *función* o usar un *método* (luego quedará mas claro cuando veamos como Python define y utiliza objetos).

Ya hemos visto algunos ejemplos de funciones (i.e. **print, input, len, ...**). Un *método* es específico a un tipo y son invocados de forma diferente.

Sea 'obj' un objeto de tipo `type` y `method` es un método asociado a al tipo `type`, entonces invocamos al método de la siguiente forma: `obj.method()`, donde, `()` puede (o no) contener algunos parámetros.

```
In [ ]: 1 c='ertl'
        2 print(len(c)) # Función
        3 c.index('r')  # Método
```

De forma practica, no hay muchas diferencias, entre llamar a una función o invocar un método. Las funciones son definidas como objetos independientes, mientras que, un método esta siempre definido dentro de un tipo de objeto (i.e. una clase, "template" del objeto) y "no existe" fuera de dicho tipo (clase del objeto). Un método, puede tener el mismo nombre, ya que cada será distinto para cada tipo de objeto. De esta manera podemos proteger ciertos nombres de variables, funciones, etc. Ejemplo:

- Podemos definir en nuestro código `count=3`, sin embargo `c.count(val)` retornara el numero de ocurrencias del parámetro `val` en la variable `c`.
- En cambio, si definimos `len=2`, luego cualquier uso de la función `len(c)` retornara un error.

```
In [ ]: 1 #len=2
        2 #len(c) #Ya no tendrá sentido invocar a la función.
        3         #Hemos 'sobrescrito' el nombre de la función len(.),
        4         #ahora el "nombre de función 'len'" esta asociado a un nombre de variable.
```

Las funciones y métodos, pueden retornar un valor (e.g. `len`) o relizar una acción (e.g. `print`). También puede modificar el valor (o valores) de algunos objetos.

```
In [ ]: 1 type(print(c)) # La función print no tiene tipo.
```

Definición de Contenedores

Informalmente, un '*contenedor*' es un "tipo de objeto" que contiene '*objetos*'. Python nos provee con distintos tipos de contenedores predefinidos.

A continuación veremos algunas propiedades de los contenedores más usados. Finalmente, haremos una rápida comparación de estos distintos tipos de estructuras de datos.

Cadena de Carácteres (Strings)

Una cadena de caracteres (string) es una secuencia ordenada de caracteres. El *i*-ésimo elemento del string `c` puede ser accessado (siempre que el índice no este fuera de rango) escribiendo `c[i]`.

```
In [ ]: 1 c='Introducción'
        2 print(c[1],c[2],c[3],c[4],c[5])# Los índices comienzan en 0.
        3 print(c[-1]) # Python nos permite el uso de 'índices negativos'.
```

En Python, todos los índices comienzan en 0.

```
In [ ]: 1 len(c) # La función len nos devuelve la longitud (cardinalidad) del contenedor.
```

```
In [ ]: 1 c='Ejemplo'
        2 print(' ',c[0],c[1],c[2],c[3],c[4],c[5],c[6])
        3 print(' ',0,1,2,3,4,5,6)
        4 print('-',7,6,5,4,3,2,1)
```

```
In [ ]: 1 c[-1222% len(c)]
```

```
In [ ]: 1 c='Ejemplo'
        2 c+=' de unión de caracteres' # El commando '+' aplicado a una cadena de caracteres
        3                               # es usado para la concatenación.
        4 c
```

Ya habrán notado que para definir cadenas de caracteres podemos usar comillas simples ' ' o comillas dobles " ". No hay diferencia entre ambas de manera formal. Sin embargo, la convención normal es la siguiente:

- Utilizamos comillas dobles cuando queremos imprimir una frase, e.g. `print` o `input`.
- Utilizamos comillas simples cuando definimos una variable del tipo cadena de caracteres.

```
In [ ]: 1 #Uso de una, dos, o tres comillas simples, ó dobles.
2 c='un string'
3 print(c)
4 c="un string" #comillas simples, ó dobles, tienen el mismo efecto.
5 print(c)
6
7 c='''un string muy muy muy muy muy muy muy muy
8 muy muy muy muy muy muy muy muy muy
9 muy muy muy muy muy muy muy muy muy largo ''' #Las triple comillas nos permite agregar c
10 #un salto de linea.
11 print(c)
12 c="""un string muy muy muy muy muy muy muy muy
13 muy muy muy muy muy muy muy muy muy
14 muy muy muy muy muy muy muy muy muy largo """
15 print(c)
```

```
In [ ]: 1 #Pertenenencia, ulitzamos el operador "in"
2
3 print('s' in c)
4 print('z' in c)
```

No se puede cambiar el valor del contenido del la variable c, mediante indexación.

```
In [ ]: 1 #La asignación de un valor utilizando el indice esta prohibido.
2 c[2]='s'
```

```
In [ ]: 1 #Pdemos accedder a un sub-conjunto de los valores del string.
2 print(c)
3 c[2:10]
```

Aquí podemos ver alunos métodos útiles (para más información utilizar la función `help(str)` o referirse a la documentación de Python). Sea c una cadena de caracteres:

- $c * n$ (donde n es un entero) retornará n copias de c concatenadas. Es equivalente a $\underbrace{c + \dots + c}_{n \text{ veces}}$.
- `c.find('car')` retorna el primer indice i (si es que existe) para el cual `c[i]='car'`.
- Pdemos comparar cadenas de caracteres. Se utilizara el orden lexicográfico.
- `c.count(st)` cuenta el número de ocurrencias del string st en c (sin contar los que se superponen).
- `c.replace(st1,st2)` retornará una copia de c donde cada ocurrencia del string 'st1' ha sido reemplazada por 'st2'.

```
In [ ]: 1 #Ejemplos
2 c=3*'11'
3 print(c)
4 c.count('11')
```

```
In [ ]: 1 c=c.replace('11','ab')
```

```
In [ ]: 1 #El método "format" nos permite realizar inseciones en un string.
2
3 c="Los enteros {1} y {0} son pares. {1} "
4
5 print(c.format(2,4)) # Inserta los valores en el string, en el lugar reservado
6 # por el número de argumento entre llaves.
7
8 c="Los enteros {} y {} son pares. {} "
9 print(c.format(0,2,'algo')) # Inserción directa (posicional) de valores en un string.
```

Tuplas

Una *tupla* es una secuencia finita de objetos, los cuales pueden ser de distintos tipos. Genralmente, definimos una tupla, escribiendo una secuencia de objetos entre *paréntesis* y separados entre *comas*. Esta es una estructura de datos *rígida*, es decir, no podemos modificar una tupla. Sin embargo, las tuplas pueden ser concatenadas, de manera

similar a la concatenación de strings. Podemos obtener la longitud de una tupla, utilizando la función `len`. También podemos acceder a los elementos de la tupla usando índices.

```
In [ ]: 1 # Definición de una tupla.
        2 x=(0,1,2,'obj',(1,2))
        3 # Tipo
        4 type(x)
```

```
In [ ]: 1 # Longitud de la tupla
        2 len(x)
```

```
In [ ]: 1 # Acceso mediante indices
        2 x[1]
```

```
In [ ]: 1 # Concatenación
        2 x=x+(2,3,4)
        3 x
```

```
In [ ]: 1 # No podemos asignar valores directamente usando indices
        2 x[1]=100
```

```
In [ ]: 1 (0,1,3)<(1,1,0)
```

No hay muchos métodos específicos asociados a las tuplas.

- `x.count(obj)` retorna el número de occurencias del objeto 'obj' en 'x'.
- `x.index(obj)` retorna el índice de la primer occurencia del objeto 'obj' en 'x'.

```
In [ ]: 1 print(x.count(2))
        2 print(x.index(2))
```

Conjuntos

Un *conjunto* es una colección finita de objetos denotados entre llaves y separados por comas. Los conjuntos tienen (deben cumplir) todas las propiedades de los conjuntos matemáticos.

```
In [ ]: 1 # Definición de un conjunto.
        2 A={1,2,4,5,6}
        3 # Tipo.
        4 type(A)
```

```
In [ ]: 1 # Al igual que en los conjuntos matemáticos, no aceptan repticiones.
        2 A={1,1,2,2,5,5,4,4,4,6}
```

```
In [ ]: 1 # Los conjuntos no soportan indexación directa.
        2 A[1]
```

```
In [ ]: 1 # Conversión de un string a un conjunto (de letras).
        2 c='será un conjunto'
        3 set(c)
```

```
In [ ]: 1 # La función len nos retorna la cardinalidad del conjunto.
        2 len(A)
```

Aquí hemos provisto una lista (no completa) de métodos útiles para el manejo de conjuntos:

- `A.intersection(B)` retorna la intersección de los conjuntos A y B.
- `A.union(B)` retorna la unión de los conjuntos A y B.
- `A.difference(B)` retorna la diferencia simetrica de conjuntos ($A \cap B^c$).
- `A.copy()` retorna una copia del conjunto A.
- Podemos comparar dos conjuntos (siguiendo el orden de inclusión).

También podemos realizar cambios directamente sobre un conjunto:

- `A.add(a)` reemplaza el conjunto `A` por $A \cup \{a\}$ (no tiene efecto si el elemento a pertenece al conjunto `A`).
- `A.discard(a)` reemplaza el conjunto `A` por `A` removiendo el elemento a (no tiene efecto si el elemento a no pertenece al conjunto `A`).
- `A.difference-update(B)` reemplazara el conjunto `A` por $A \cap B^c$.

```
In [ ]: 1 A={1,2,4}
        2 B=A.copy()
        3 print(A,B)
        4 B.discard(2)
        5 print(A,B)
```

Una manera útil de definir conjuntos matemáticamente es por comprensión (en vez de definirlos por "extensión") i.e. $B := \{f(x) \mid x \in A\}$. En Python lo escribimos en la siguiente forma:

$$B = \{funcion(x) \text{ for } x \text{ in } A\}$$

```
In [ ]: 1 # Ejemplo de cómo definir un conjunto por comprensión.
        2 B={x for x in c}
        3 B
```

Diccionarios.

Un diccionario o "tabla asociativa" es un contenedor muy particular y de amplio uso (están optimizados al igual que cualquier tipo "hashable" en Python). Es una colección de items "llave:valor" donde la "llave" y el "valor" pueden ser cualquier tipo de objeto, y los escribimos de la siguiente forma:

$$c = \{l_1 : v_1, l_2 : v_2, \dots\}$$

```
In [ ]: 1 dicc1={'Jean Paul':'jeanpaul@mail.com',\
        2 'Fanny':'fanny@mail.com',\
        3 'Robert':'robert@mail.com',\
        4 'Stephanie': (6812424239),\
        5 0:2}
        6
        7 # Tipo
        8 print(type(dicc1))
```

- La función `len` nos devuelve la cardinalidad.
- No podemos indexarlos de manera habitual, es decir de manera posicional, deberemos usar las llaves de diccionario para retornar los valores correspondientes.
- Es posible asignar nuevos valores a una las llaves.

```
In [ ]: 1 dicc1={'Jean Paul':'jeanpaul@mail.com',\
        2 'Fanny':'fanny@mail.com',\
        3 'Robert':'robert@mail.com',\
        4 'Stephanie': (6812424239),\
        5 0:2}
        6 # Es posible obtener la cardinalidad.
        7 len(dicc1)
        8 # Los diccionarios no pueden ser indexados posicionalmente. Debemos proveer la llave c
        9 dicc1['Jean Paul']
       10 dicc1[0]
```

```
In [ ]: 1 # Tipo
        2 type(dicc1)
```

```
In [ ]: 1 dicc1={'Jean Paul':'jeanpaul@mail.com',\
2         'Fanny':'fanny@mail.com',\
3         'Robert':'robert@mail.com',\
4         'Stephanie': (6812424239),\
5         0:2}
6 # Podemos cambiar los valores asociados a un llave mediante asignación.
7 dicc1['Stephanie']='stephanie@mail.com'
8 print(dicc1)
```

```
In [ ]: 1 # Cual es el efecto de tener una llave duplicada?
2 dicc2={0:7,'x':'x@mail.lu', 0:3}
3 print(dicc2)
4 # La segunda (o última) llave definida, sobrescribira el valor anterior.
```

```
In [ ]: 1 # Podemos convertir un diccionario a una tupla
2 tuple(dicc1)
3 # No es posible en el sentido contrario (de tupla a diccionario).
4 dict(dicc1)
```

Aquí tenemos algunos métodos útiles para usar diccionarios. Sea `dic` un diccionario.

- `dic.item()` retorna una *lista* de los valores en el diccionario (las listas están definidas en la siguiente sección).
- `dic.keys()` solo retorna los valores de las llaves del diccionario.
- `dic.values()` solo retorna los valores del diccionario.
- `dic.copy()` retorna una copia del diccionario `dic`.
- `dic.pop(key)` retorna y quita del diccionario el valor que referido por la llave `key`
- `dic.popitem()` retorna y quita del diccionario el último item ingresado/añadido.
- `dic.update(newdic)` actualiza `dic` con los valores de otro diccionario `newdic`, solo para las llaves de `newdic` que no están en `dic`.

- `dic[key]=value`

Asignará un nuevo valor a la llave `key`, solo si la llave ya pertenece al diccionario. En caso de que la llave no exista en el diccionario, añadirá el nuevo par (`key:value`) al diccionario.

```
In [ ]: 1 dicc1={'Jean Paul':'jeanpaul@mail.com',\
2         'Fanny':'fanny@mail.com',\
3         'Robert':'robert@mail.com',\
4         'Stephanie': (6812424239),\
5         0:2}
6 dicc1['Stephanie']='stephanie@mail.com'
7 dicc1['Carl']='carl@mail.com'
8 print(dicc1)
```

```
In [ ]: 1 dicc1={'Jean Paul':'jeanpaul@mail.com',\
2         'Fanny':'fanny@mail.com',\
3         'Robert':'robert@mail.com',\
4         'Stephanie': (6812424239),\
5         0:2}
6 dicc2={0:7,'x':'x@mail.com',0:3}
7 print(dicc1,dicc2)
8 dicc1.update(dicc2)
9 print(dicc1)
```

Listas

Una lista es un contenedor ordenado de objetos (pueden ser de distinto tipo). Lo definiremos ente *corchetes* y los objetos separados por *comas*.

$$L = [x, y, \dots]$$

```
In [ ]: 1 # Definición
        2 L=[2,3,4]
        3 # Tipo
        4 type(L)
```

```
In [ ]: 1 # También podemos definir Listas por comprensión (al igual que los conjuntos).
        2 L=[x**2 for x in range(0,9)]
        3 L
```

Podemos utilizar los índices de distintas maneras para poder acceder mas convenientemente a partes de una lista. Esta notación funciona para cualquier tipo de contenedor ordenado (i.e. strings, tuples, etc).

- `L [ i ]` retorna el i -ésimo elemento de la lista `L`.
- `L [ i : j ]` retorna los elementos desde el i -ésimo (incluido) hasta el j -ésimo (excluido). El resultado tiene el mismo tipo que `L`.
- `L [ i : ]` es equivalente a: `L [ i : len(L) ]`.
- `L [ : j ]` es equivalente a: `L [ 0 : j ]`.
- `L [ i : : paso ]` retorna la lista de elemntos desde la i -ésima posición progresando sobre los índices de acuerdo a `paso`.

También podemos concatenar listas usando el operador '+'

```
In [ ]: 1 # Cambiar el i-ésimo elemento.
        2 L=[1,2,3,4,5,6]
        3 L[3]=12
        4 print(L)
```

```
In [ ]: 1 # Podemos acceder a la lista usando indices.
        2 L=[1,2,3,4,5,6]
        3 L[2::3]
```

```
In [ ]: 1 # Concatenación de Listas
        2 L=[1,2,3,4,5,6]
        3 M=['caballo','perro']
        4 N= L + M
        5 print(N)
```

Aquí hay una breve lista de los métodos asociados al tipo *Lista*:

- `L.count(obj)` retorna el número de ocurrencias del objeto `obj` en la `L`.
- `L.index(value)` retorna el primer índice i para el cual `L [ i ] = value`.
- `L.insert(i, obj)` inserta el objeto `obj` en la i -ésima posición, moveindo el resto de los contenidos de la lista hacia la derecha.
- `L.remove(value)` remueve de `L` la primer ocurrencia de `value`.
- `L.pop(index)` retorna el valor indexado y lo remueve de `L`.
- `L.reverse()` escribe la lista `L` del revez (cambia `L`).
- `L.sort(L)` (re)ordena `L` de acuerdo al orden lexicográfico. Los elementos deben ser del mismo tipo.

```
In [ ]: 1 L=['1','5','2','caballo','3']
        2 L.sort()
        3 print(L)
```

Rangos

Los Rangos, son tipos especiales de contenedores. Generalmente, construiremos un rango de la siguiente forma `range` (comienzo, fin, paso).

Este commando generara un *rango* de números enteros desde comienzo (incluido) hasta stop (excluido) tomando en cuenta la "longitud de salto" definida por `paso`.

También pordemos invocar a `range` (comienzo, fin). y el '`paso`' será por defecto 1, también podemos invocar `range` (comienzo). donde el '`comienzo`' será por defecto 0.

```
In [ ]: 1 # Por que no vemos un resultado?
        2 range(1,100,2)
```

El rango no es un objeto **per se**, podemos tratarlo como una '*potencial*' lista de enteros. Sin embargo, también es posible chequear que 'algo' esta o pertenece a cierto rango, i.e. 'algo' **in range** (comienzo, fin).

Diferencias y relaciones entre estas Estructuras de Datos

- Un diccionario es una forma conveniente de almacenar/actualizar/eliminar información sobre llaves específicas. Sin embargo, es un objeto de mayor complejidad y debe ser utilizado con cuidado.
- Una cadena de caracteres es un objeto específico. Es una forma simple se comunicarse con el usuario, usando el método `format`.
- Los strings y la tuplas, son tipos inmutables. No existen métodos definidos para cambiar sus valores internos.

```
In [ ]: 1 # Es posible cambiar de 'str' a tipo 'tuple'.
        2 x=(1,2,3)
        3 x=x+x
        4 print(x)
```

- Los Conjuntos, Diccionarios y Listas son tipos "mutables"

Existen diversos métodos para manipularlos.

Debemos tener cuidado cuando utilizamos la asignación en tipos mutables, ya en generalmente sobrescribiremos el o los valores.

Para copiar un objeto complejo debemos usar el método `copy` en ves del simbolo `=`.

- Los objetos "mutables" tiene un tiempo de acceso mayor (son un poco más lentos).

```
In [ ]: 1 # Efecto del método copy().
        2 L=[1,2]
        3 M=[3,4]
        4 biglist=[L,M]
        5 copybiglist=biglist.copy()
        6 copybiglist.append(1)
        7 print(biglist,copybiglist)
        8 copybiglist[0]=[1,3]
        9 print(biglist,copybiglist)
```

- La desventaja de usar tipos "mutables" es el acceso más lento a los datos.

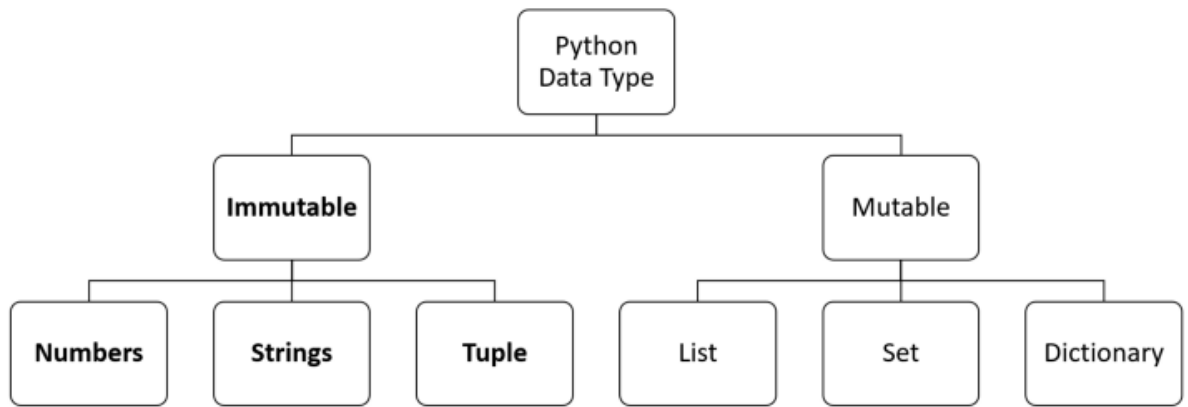
```
In [ ]: 1 # Efectos de los cambios de tipo (forzados).
        2 L=[1,2,1,2,1,3,4,2,3]
        3 c=list(set(L))
        4 print(c)
```

Punteros

En otros lenguajes, como C o C++, existe un tipo de variable especial llamado puntero, que se comporta como una referencia a una variable, **como es el caso de las variables mutables**.

En Python *no hay punteros* (como los de C o C++), pero todas las variables son referencias a objetos, es decir, la *dirección* de la porción de memoria, en donde el objeto está almacenado. De este modo cuando se asigna una variable a otra, lo que se está asignando es la dirección de la porción de memoria guarda el objeto. Si esa porción de memoria cambia, el cambio se puede ver en todas las variables que apuntan a esa porción.

Resumen Clasificación



Ciclos y Repeticiones

Ciclos while (mientras).

Un bucle **while** se representa de la siguiente manera:

```

In [ ]: 1 N=int(input("Contaremos hasta : "))
        2 p=0
        3 while p<N: # Definimos la condición sobre la que ejecutara el ciclo.
        4     p+=1 # Hacemos algo dentro del ciclo.
        5     print(p,end=" ") # Hacemos otra cosa.
  
```

Al igual que para el *condicional*, debemos terminar la línea con la instrucción **while** con ':', luego debemos indentar el bloque de comandos que será repetido en el ciclo.

En general, cuando definimos un ciclo **while**, debemos asegurarnos que el ciclo termine en algún momento. En caso de omitir una terminación segura del ciclo, el programa nunca ejecutara los comandos que estén a continuación del ciclo, para retomar el control, deberemos reiniciar el kernel.

Ciclos for (para).

Cuando debemos realizar una tarea repetitiva, es muy conveniente utilizar los ciclos **for**. La forma estándar de escribirlos es la siguiente:

for variable **in** 'contenedor':

seguido de un salto de línea y luego las instrucciones a ser repetidas. Como cualquier instrucción con :, necesitaremos indentar.

variable es cualquier nombre de variable (no protegido), y 'contenedor' es un objeto contenedor.

```

In [ ]: 1 # Ejemplo de un ciclo
        2 for x in range(0,20):
        3     print(x,end=" ")
  
```

Debemos tener cuidado que **range** no nos retorna un tipo de datos conveniente. Igualmente puede ser (o será) convertido al tipo lista. Si utilizáramos el tipo lista directamente, la performance se verá reducida.

```

In [ ]: 1 # Comparación con los ciclos "mientras"
        2 x=0
        3 while x<20:
        4     print(x,end=" ")
        5     x+=1
  
```

```
In [ ]: 1 # Uso de ciclos for, para distintos tipos de contenedores.
        2
        3 for x in {0,1,2}: # Conjunto
        4     print(x,end=' ')
        5 print('')
        6 for x in 'AlgoritmosEstructurasDatos': # Strings
        7     print(x,end=' ')
        8 print('')
        9 for x in list('AlgoritmosEstructurasDatos'): # Listas
        10     print(x,end=' ')
        11
```

Algunas veces, cuando recorremos una lista `L` utilizando un ciclo `for` nos gustaría poder ver los valores y los índices.

```
In [ ]: 1 # Un modo sencillo de hacerlo.
        2 L=['1','5','2','caballo','3']
        3 for index in range(0,len(L)):
        4     print("L[{}]={}".format(index,L[index]))
```

También podemos utilizar la siguiente notación reducida `for index,variable in L` : En este caso, tendremos `L[index]=variable` durante la ejecución del ciclo.

```
In [ ]: 1 for index,variable in enumerate(L): # Usamos la funcion enumerate para acceder a las
        2     print("L[{}]={}".format(index,variable))
```

Podemos interrumpir el ciclo `for` (respectivamente para `while`) utilizando el comando `break`.

```
In [ ]: 1 for x in range(0,9):
        2     if x>6:
        3         break
        4     print(x,end=" ")
```

```
In [ ]: 1 for y in range(0,9):
        2     for x in range(0,9):
        3         if x>6:
        4             break # Solo interrumpimos el ciclo en "donde estamos".
        5     print(10*y+x,end=" ")
```